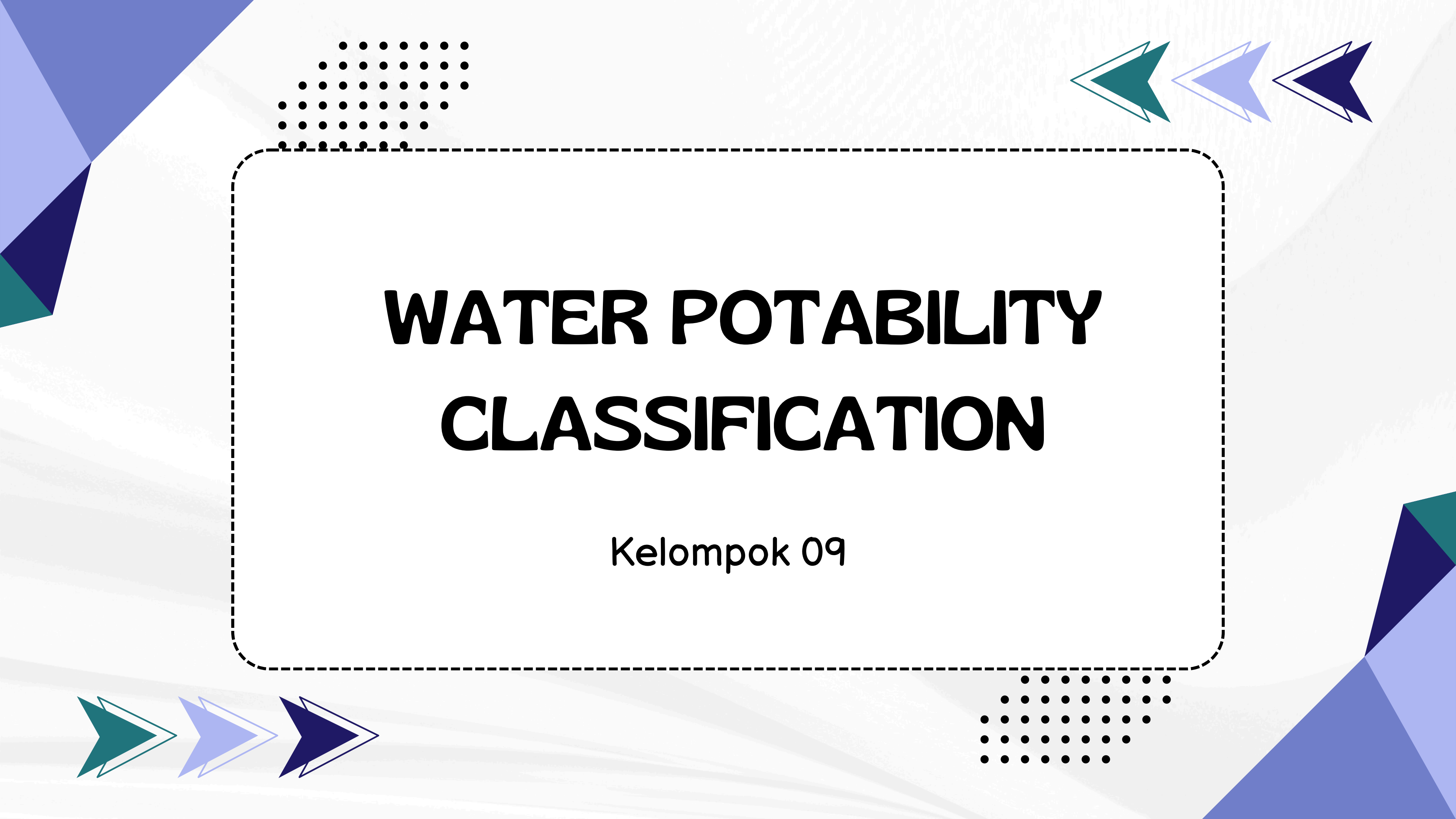




# **WATER POTABILITY CLASSIFICATION**

Kelompok 09

# ANGGOTA

Fazhira Rizky Harmayani

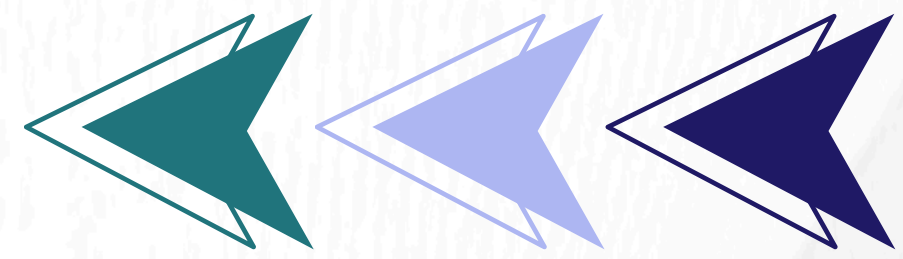
Cut Dahliana

Riska Haqika Situmorang

Naufal Aqil

Hidayat Nur Hakim





# Pemahaman Dataset

## Nama Dataset :

Dataset water\_potability.csv

## Sumber Kaggle :

<https://www.kaggle.com/datasets/adityakadiwal/water-potability/data>

## Deskripsi Dataset :

Dataset water\_potability.csv adalah kumpulan data yang sangat berguna untuk analisis klasifikasi dalam bidang kesehatan lingkungan dan pengolahan air. Dataset ini mencakup 3.276 sampel air dari berbagai sumber dan dilengkapi dengan 9 fitur numerik yang digunakan untuk menentukan kelayakan air minum.

Label target adalah kolom Potability, dengan nilai:

- 1 → Tidak layak minum
- 0 → Layak minum

## Jumlah Data :

Dataset ini terdiri dari 3.276 baris dan 10 kolom yang merepresentasikan berbagai parameter kualitas air. Setiap baris menunjukkan sampel air yang diuji, sementara fitur-fitur numeriknya mencakup nilai-nilai kimia dan fisik dari air tersebut. Fitur-fitur tersebut meliputi ph (tingkat keasaman air), Hardness (jumlah kalsium dan magnesium), Solids (total zat padat terlarut), Chloramines (disinfektan kombinasi klorin dan amonia), Sulfate (senyawa alami dari batuan dan tanah), Conductivity (kemampuan air menghantarkan listrik), Organic\_carbon (kandungan karbon organik), Trihalomethanes (senyawa kimia dari proses disinfeksi), dan Turbidity (kejernihan air berdasarkan partikel tersuspensi). Label target Potability menunjukkan apakah air tersebut layak minum (0) atau tidak layak minum (1), menjadikan dataset ini relevan untuk klasifikasi dalam bidang kesehatan lingkungan.

# Exploratory Data Analysis (EDA)

- Memuat dan Menampilkan Informasi Awal Dataset

```
water_df = pd.read_csv('./data/water_potability.csv')
```

Dataframe has 3276 Rows, 10 Columns  
\*\*\*\*\*

|   | ph       | Hardness   | Solids       | Chloramines | Sulfate    | Conductivity | Organic_carbon | Trihalomethanes | Turbidity | Potability |
|---|----------|------------|--------------|-------------|------------|--------------|----------------|-----------------|-----------|------------|
| 0 | nan      | 204.890455 | 20791.318981 | 7.300212    | 368.516441 | 564.308654   | 10.379783      | 86.990970       | 2.963135  | 0          |
| 1 | 3.716080 | 129.422921 | 18630.057858 | 6.635246    | nan        | 592.885359   | 15.180013      | 56.329076       | 4.500656  | 0          |
| 2 | 8.099124 | 224.236259 | 19909.541732 | 9.275884    | nan        | 418.606213   | 16.868637      | 66.420093       | 3.055934  | 0          |
| 3 | 8.316766 | 214.373394 | 22018.417441 | 8.059332    | 356.886136 | 363.266516   | 18.436524      | 100.341674      | 4.628771  | 0          |
| 4 | 9.092223 | 181.101509 | 17978.986339 | 6.546600    | 310.135738 | 398.410813   | 11.558279      | 31.997993       | 4.075075  | 0          |

Kolom yang ada meliputi berbagai parameter kualitas air dan target 'Potability'

- Melihat Sample Data dan Statistik Deskriptif

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 3276 entries, 0 to 3275  
Data columns (total 10 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
0   ph           2785 non-null   float64  
1   Hardness     3276 non-null   float64  
2   Solids       3276 non-null   float64  
3   Chloramines  3276 non-null   float64  
4   Sulfate      2495 non-null   float64  
5   Conductivity 3276 non-null   float64  
6   Organic_carbon 3276 non-null float64  
7   Trihalomethanes 3114 non-null float64  
8   Turbidity    3276 non-null   float64  
9   Potability   3276 non-null   int64  
dtypes: float64(9), int64(1)  
memory usage: 256.1 KB
```

Semua kolom memiliki tipe data float64 dan terdapat missing values di beberapa kolom

- Statistik Deskriptif

|                 | count       | mean         | std         | min        | 25%          | 50%          | 75%          | max          |
|-----------------|-------------|--------------|-------------|------------|--------------|--------------|--------------|--------------|
| ph              | 2785.000000 | 7.080795     | 1.594320    | 0.000000   | 6.093092     | 7.036752     | 8.062066     | 14.000000    |
| Hardness        | 3276.000000 | 196.369496   | 32.879761   | 47.432000  | 176.850538   | 196.967627   | 216.667456   | 323.124000   |
| Solids          | 3276.000000 | 22014.092526 | 8768.570828 | 320.942611 | 15666.690297 | 20927.833607 | 27332.762127 | 61227.196008 |
| Chloramines     | 3276.000000 | 7.122277     | 1.583085    | 0.352000   | 6.127421     | 7.130299     | 8.114887     | 13.127000    |
| Sulfate         | 2495.000000 | 333.775777   | 41.416840   | 129.000000 | 307.699498   | 333.073546   | 359.950170   | 481.030642   |
| Conductivity    | 3276.000000 | 426.205111   | 80.824064   | 181.483754 | 365.734414   | 421.884968   | 481.792304   | 753.342620   |
| Organic_carbon  | 3276.000000 | 14.284970    | 3.308162    | 2.200000   | 12.065801    | 14.218338    | 16.557652    | 28.300000    |
| Trihalomethanes | 3114.000000 | 66.396293    | 16.175008   | 0.738000   | 55.844536    | 66.622485    | 77.337473    | 124.000000   |
| Turbidity       | 3276.000000 | 3.966786     | 0.780382    | 1.450000   | 3.439711     | 3.955028     | 4.500320     | 6.739000     |
| Potability      | 3276.000000 | 0.390110     | 0.487849    | 0.000000   | 0.000000     | 0.000000     | 1.000000     | 1.000000     |

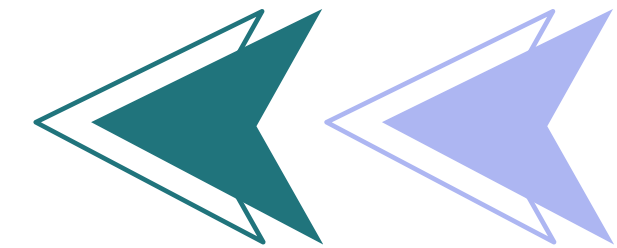
- Visualisasi dengan gradient warna menunjukkan variasi standar deviasi
- Mean dan median ditampilkan dengan bar chart berwarna merah muda

- Cek Nilai Unik, Missing Values, dan Duplikasi

```
Jumlah Nilai Unik per Kolom:  
ph           2785  
Hardness     3276  
Solids       3276  
Chloramines  3276  
Sulfate      2495  
Conductivity 3276  
Organic_carbon 3276  
Trihalomethanes 3114  
Turbidity    3276  
Potability   2  
dtype: int64
```

Mayoritas parameter dalam dataset memiliki nilai unik maksimal (3276), yang menunjukkan presisi pengukuran yang tinggi. Namun, pH dan Sulfate memiliki nilai unik yang lebih sedikit, kemungkinan disebabkan oleh adanya missing values.





- **Mengecek jumlah missing values**

```
Missing Values dalam Dataset:
ph          491
Hardness    0
Solids      0
Chloramines 0
Sulfate     781
Conductivity 0
Organic_carbon 0
Trihalomethanes 162
Turbidity   0
Potability  0
dtype: int64
```

pH memiliki 491 missing values, Sulfate memiliki 781 missing values, dan Trihalomethanes memiliki 162 missing values.

- **Menampilkan data duplikat**

```
Data Duplikat dalam Dataset:
0
```

Tidak di temukan data duplikat

- **Menghapus Nilai Kosong (Missing Values)**

```
ph          0
Hardness    0
Solids      0
Chloramines 0
Sulfate     0
Conductivity 0
Organic_carbon 0
Trihalomethanes 0
Turbidity   0
Potability  0
dtype: int64
```

Semua fitur memiliki 0 nilai kosong. Dataset sudah bersih.

- **Menyeimbangkan Distribusi Kelas (Upsampling)**

```
# Menyeimbangkan kelas (upsampling class 1)
from sklearn.utils import resample, shuffle
df_0 = water_df[water_df['Potability'] == 0]
df_1 = water_df[water_df['Potability'] == 1]
df_1_upsampled = resample(df_1, replace=True, n_samples=len(df_0), random_state=42)
df_balanced = shuffle(pd.concat([df_0, df_1_upsampled]), random_state=42)
```

Membuat model tidak berat sebelah dan adil dalam memprediksi kedua kelas.

# Implementasi Model

- Pisahkan fitur dan target

```
# === DEFINISI FITUR DAN TARGET ===  
X = df_balanced.drop('Potability', axis=1)  
y = df_balanced['Potability']
```

- Bagi Data menjadi Training & Testing (80%-20%)

```
# === SPLIT DATA TRAIN-TEST ===  
X_train, X_test, y_train, y_test = train_test_split(  
    X_scaled, y, test_size=0.1, random_state=42, stratify=y)
```

| Model               | Akurasi Awal                             |
|---------------------|------------------------------------------|
| Logistic Regression | 51% ❌ (tidak lebih baik dari tebak acak) |
| KNN                 | 75% ✅ (sudah cukup bagus)                |
| Naive Bayes         | 53% ❌                                    |
| Decision Tree       | 81% ✅ (paling bagus sebelum tuning)      |

- Evaluasi Awal Sebelum Tuning

```
# === INISIALISASI MODEL (SEBELUM TUNING) ===  
models = {  
    'Logistic Regression': LogisticRegression(),  
    'KNN': KNeighborsClassifier(),  
    'Naive Bayes': GaussianNB(),  
    'Decision Tree': DecisionTreeClassifier()  
}  
  
print("=== Akurasi Sebelum Tuning ===")  
for name, model in models.items():  
    model.fit(X_train, y_train)  
    y_pred = model.predict(X_test)  
    print(f"{name}: {accuracy_score(y_test, y_pred):.2f}")
```

```
=== Akurasi Sebelum Tuning ===  
Logistic Regression: 0.51  
KNN: 0.75  
Naive Bayes: 0.53  
Decision Tree: 0.81
```

- Logistic Regression dan Naïve Bayes belum optimal.
- KNN dan Decision Tree menunjukkan potensi kuat untuk klasifikasi data ini.

## • Hyperparameter Tuning (GridSearchCV)

```
# === INISIALISASI UNTUK TUNING ===
results = {}
best_models = {}
```

```
# === LOGISTIC REGRESSION TUNING ===
print("\n🐡 Sedang melakukan tuning pada model Logistic Regression ...")
lr = LogisticRegression()
param_lr = {'C': [0.01, 0.1, 1, 10], 'solver': ['liblinear', 'lbfgs']}
grid_lr = GridSearchCV(lr, param_lr, cv=5)
grid_lr.fit(X_train, y_train)
best_models['Logistic Regression'] = grid_lr.best_estimator_
results['Logistic Regression'] = grid_lr.best_score_

# === KNN TUNING ===
print("\n🐡 Sedang melakukan tuning pada model KNN ...")
knn = KNeighborsClassifier()
param_knn = {'n_neighbors': np.arange(1, 50), 'weights': ['uniform', 'distance']}
grid_knn = GridSearchCV(knn, param_knn, cv=5)
grid_knn.fit(X_train, y_train)
best_models['KNN'] = grid_knn.best_estimator_
results['KNN'] = grid_knn.best_score_

# === NAIVE BAYES (TIDAK PERLU TUNING) ===
print("\n🐡 Tidak melakukan tuning Naive Bayes ...")
nb = GaussianNB()
nb.fit(X_train, y_train)
best_models['Naive Bayes'] = nb
results['Naive Bayes'] = nb.score(X_train, y_train)

# === DECISION TREE TUNING ===
print("\n🐡 Sedang melakukan tuning pada model Decision Tree ...")
dt = DecisionTreeClassifier()
param_dt = {
    'criterion': ['gini', 'entropy'],
    'max_depth': np.arange(1, 50),
    'min_samples_leaf': [1, 2, 4, 5, 10, 20, 30, 40, 80, 100]
}
grid_dt = GridSearchCV(dt, param_dt, cv=5)
grid_dt.fit(X_train, y_train)
best_models['Decision Tree'] = grid_dt.best_estimator_
results['Decision Tree'] = grid_dt.best_score_
```

Untuk meningkatkan performa model, dilakukan pencarian kombinasi parameter terbaik dengan Cross Validation (CV 5-fold).

- GridSearchCV: Melatih model beberapa kali dengan berbagai kombinasi parameter, lalu memilih yang terbaik.

# Evaluasi Model

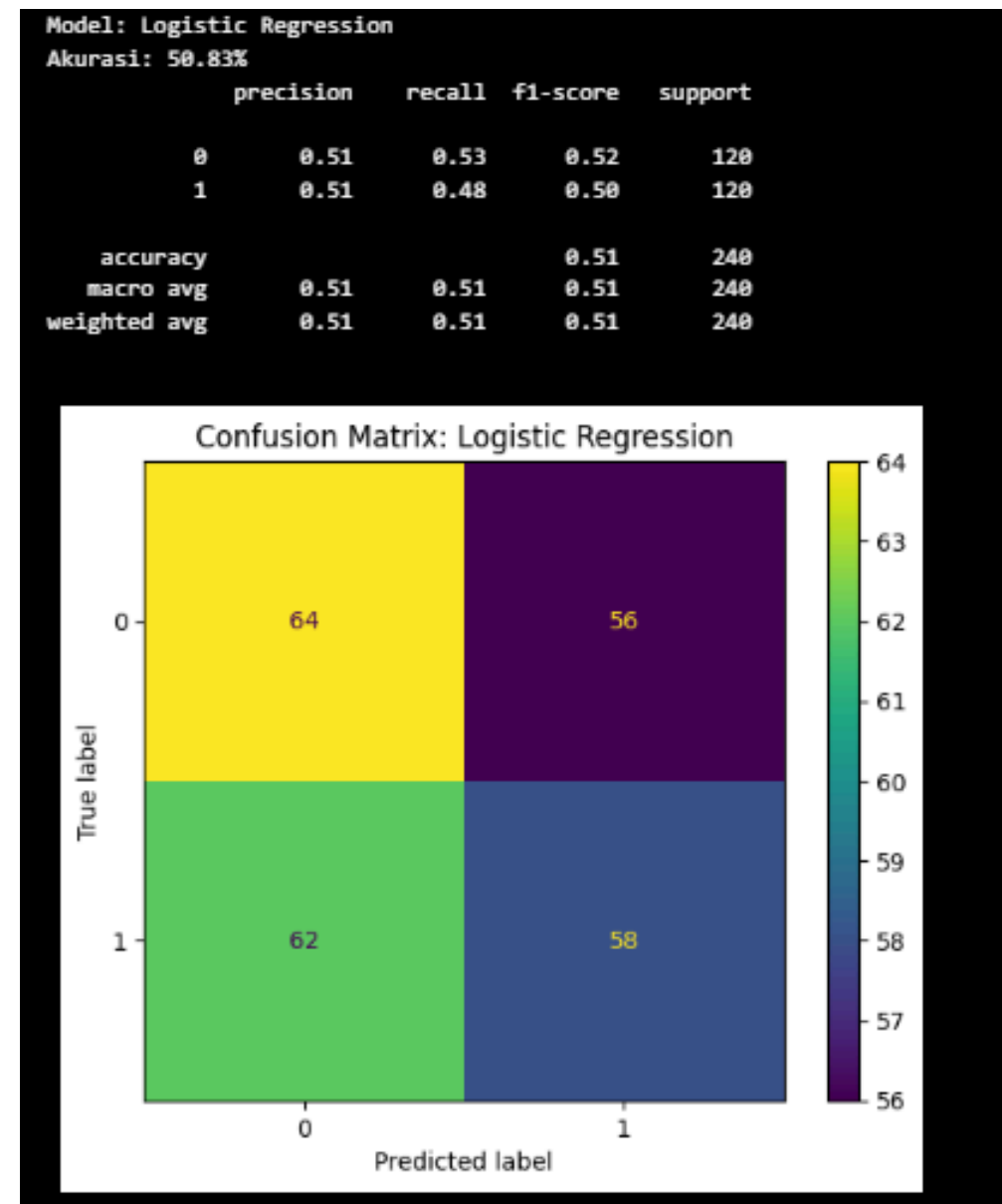
- **Evaluasi Akhir Setelah Tuning**

Setelah proses tuning, model diuji ulang menggunakan data test dan dievaluasi menggunakan beberapa metrik, yaitu akurasi (persentase prediksi yang benar), precision (proporsi prediksi positif yang benar-benar positif), recall (proporsi data positif yang berhasil ditemukan), dan F1-score (rata-rata harmonis antara precision dan recall).

```
# Evaluasi semua model di test set
print("\n📊 Evaluasi Semua Model di Test Set:")
for name, model in best_models.items():
    print(f"\nModel: {name}")
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred) * 100
    print(f"Akurasi: {acc:.2f}%")
    print(classification_report(y_test, y_pred))

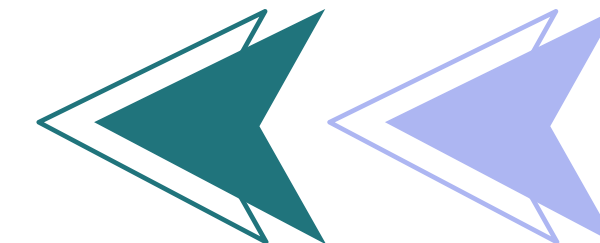
    cm = confusion_matrix(y_test, y_pred)
    ConfusionMatrixDisplay(cm, display_labels=[0, 1]).plot()
    plt.title(f"Confusion Matrix: {name}")
    plt.show()
```

- **Evaluasi Logistic Regression**

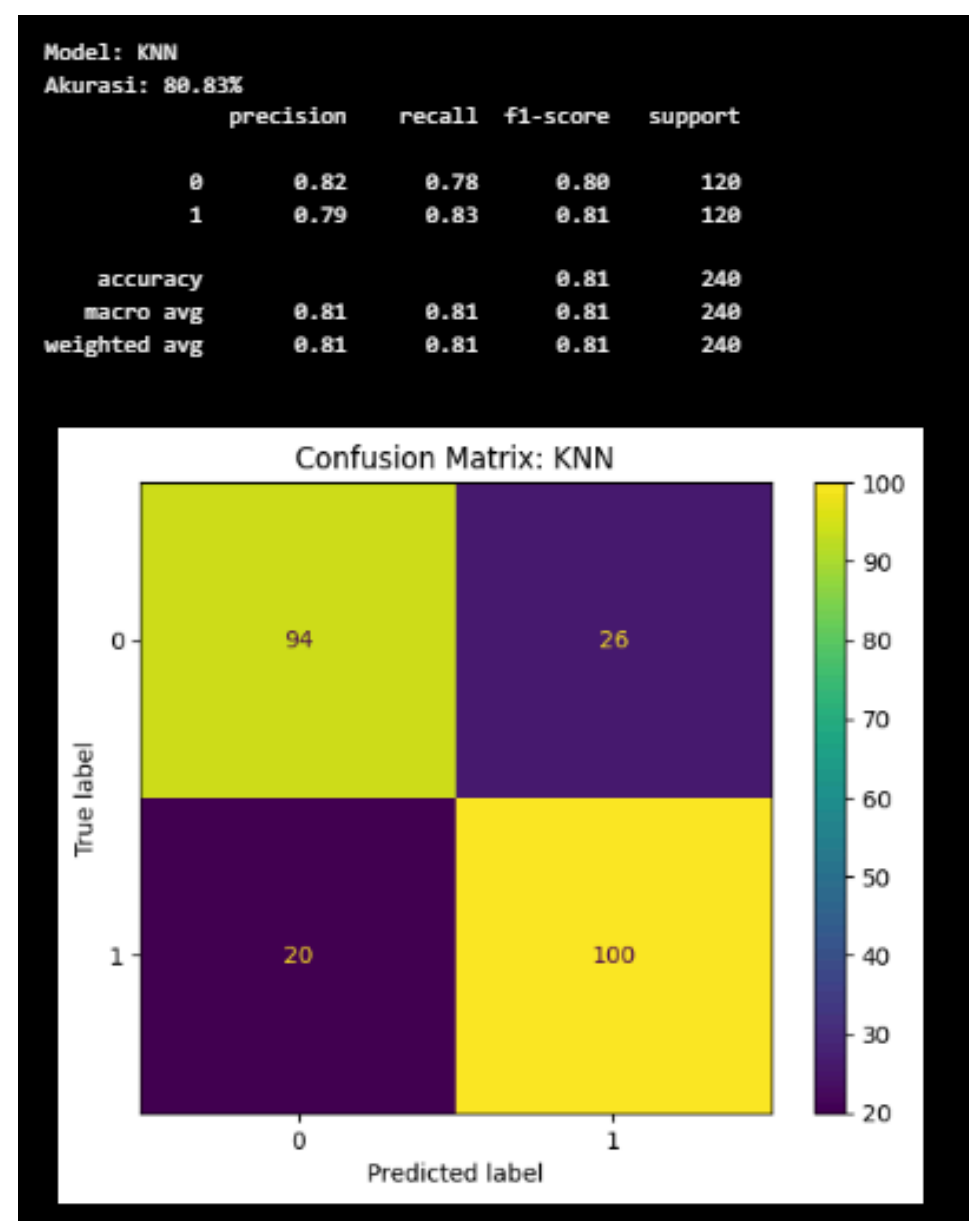


Model Logistic Regression menghasilkan akurasi 58% dengan banyak kesalahan klasifikasi antar kelas, terlihat dari confusion matrix. Precision dan recall rendah, menunjukkan model kurang efektif untuk dataset ini.



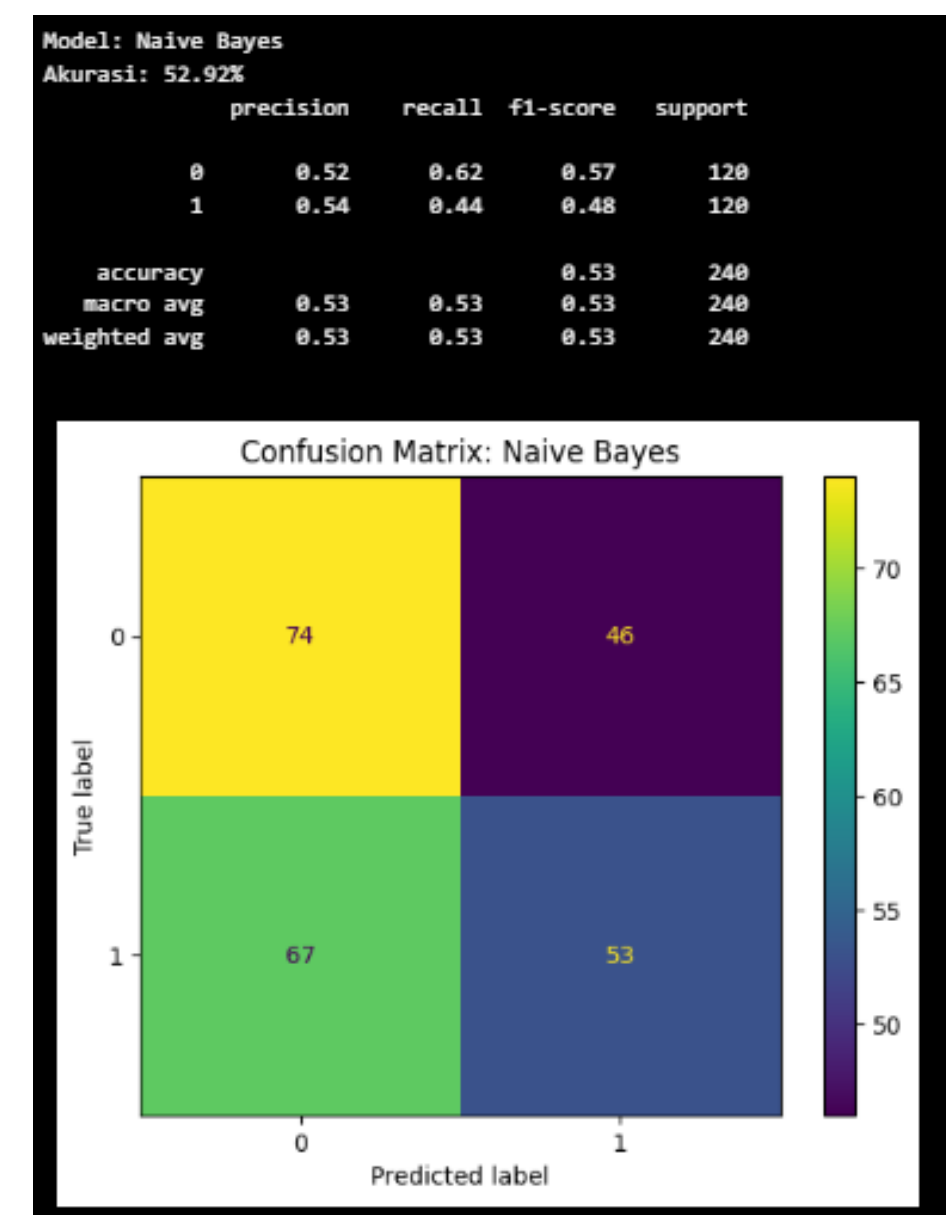


- **Evaluasi K-Nearest Neighbors (KNN)**

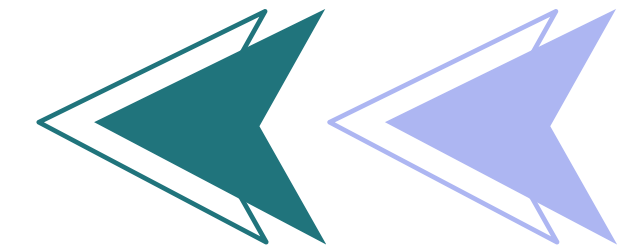


Model KNN menunjukkan performa terbaik dengan akurasi 80%. Confusion matrix menunjukkan prediksi yang cukup akurat untuk kedua kelas, menandakan bahwa model ini efektif mengenali pola dalam data.

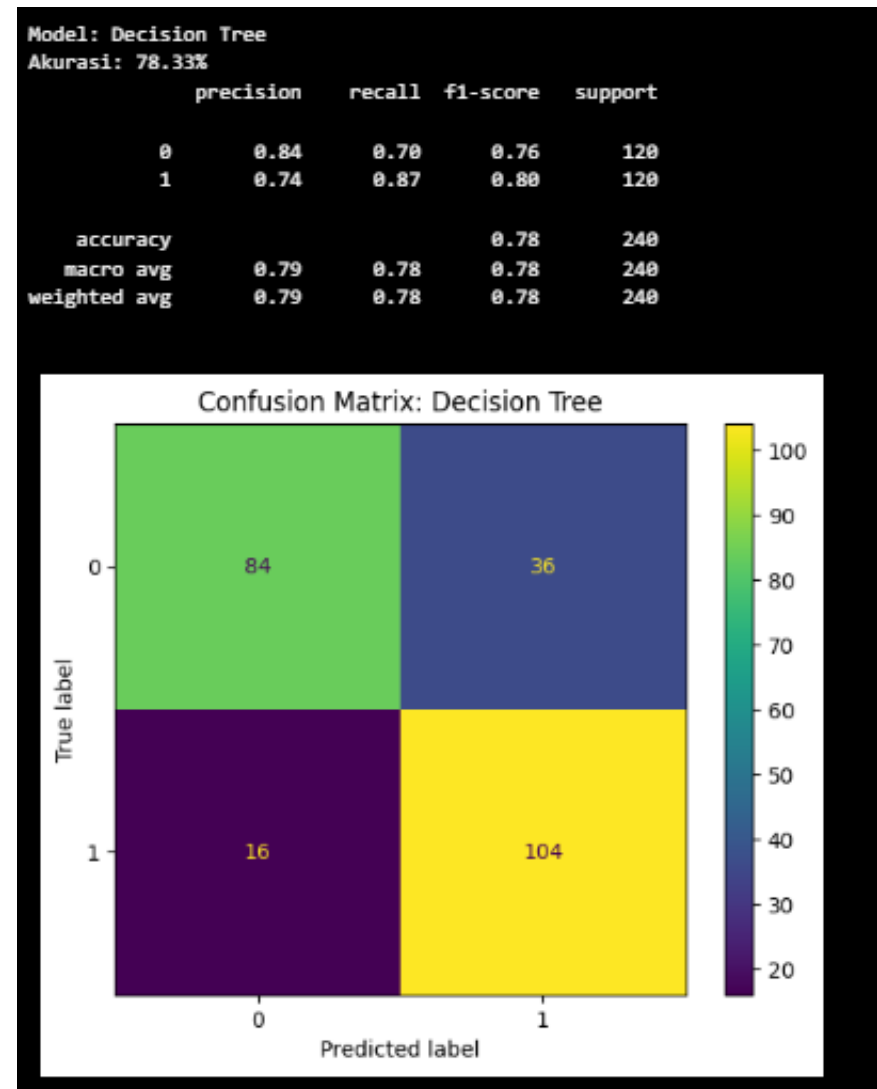
- **Evaluasi Naïve Bayes**



Naïve Bayes hanya mencapai akurasi 55% dan menghasilkan banyak kesalahan klasifikasi. Hal ini menunjukkan bahwa model ini kurang mampu menangani distribusi data yang kompleks atau tidak normal.



- **Evaluasi Decision Tree**



Model Decision Tree memiliki akurasi 76,33% dengan kinerja cukup seimbang. Confusion matrix menunjukkan 84 dan 104 prediksi benar untuk masing-masing kelas, namun masih ada kesalahan klasifikasi antar kelas yang perlu diperbaiki.

- **Output Evaluasi**

| Model               | Akurasi Test | Kesimpulan                                 |
|---------------------|--------------|--------------------------------------------|
| Logistic Regression | 50.83%       | ⚠️ Masih lemah dan tidak stabil            |
| KNN                 | 80.83%       | ✅ Akurat dan seimbang                      |
| Naive Bayes         | 52.92%       | ⚠️ Performa kurang, recall buruk           |
| Decision Tree       | 78.33%       | ✅ Stabil, walau overfit perlu diperhatikan |

Tabel menunjukkan bahwa KNN memiliki akurasi terbaik (80,83%) dan performa seimbang. Decision Tree cukup stabil dengan akurasi 78,33%, meski berpotensi overfit. Naive Bayes dan Logistic Regression menunjukkan akurasi rendah, menandakan performa yang kurang baik dan tidak stabil.

# Analisis Hasil

Hasil akhir menunjukkan KNN sebagai model terbaik dengan akurasi 80% setelah tuning GridSearchCV. KNN efektif meski data tidak teratur. Decision Tree juga layak dipertimbangkan, sementara Logistic Regression dan Naïve Bayes kurang cocok. Diagram batang menunjukkan urutan performa: KNN, Decision Tree, Logistic Regression, dan Naïve Bayes.

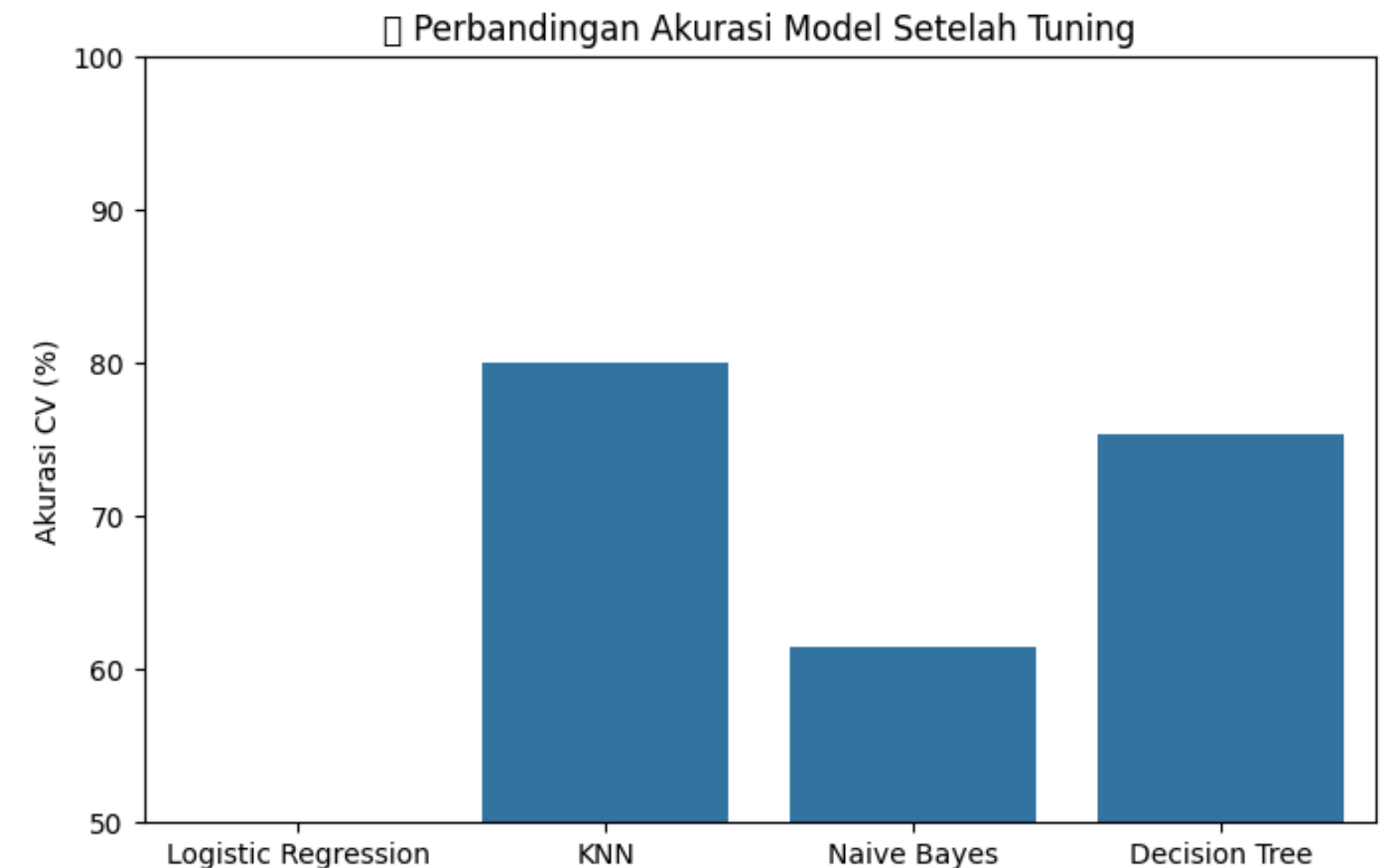
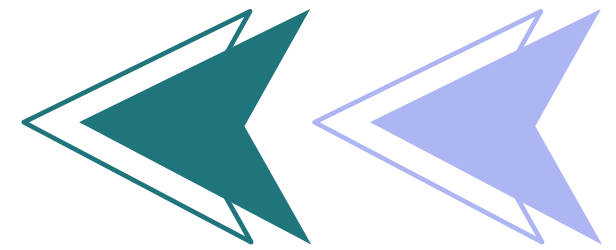
- **Grafik Akurasi Cross-Validation**

```
# Grafik perbandingan akurasi (CV)
plt.figure(figsize=(8,5))
sns.barplot(x=list(results.keys()), y=[v*100 for v in results.values()])
plt.ylabel("Akurasi CV (%)")
plt.title("📊 Perbandingan Akurasi Model Setelah Tuning")
plt.ylim(50, 100)
plt.show()
```

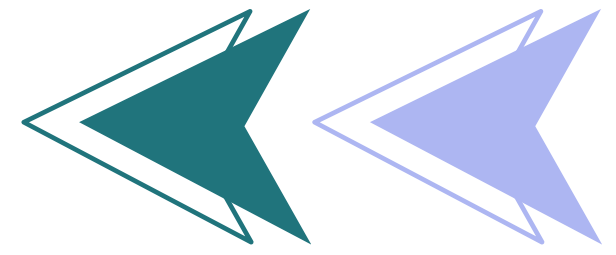
- **Model Terbaik: KNN**

```
# Model terbaik
best_model_name = max(results, key=results.get)
print(f"\n✅ Model terbaik berdasarkan cross-validation adalah: {best_model_name} dengan akurasi {results[best_model_name]*100:.2f}%")
```

KNN adalah model terbaik dengan akurasi CV 80.00%, menunjukkan bahwa pendekatan berbasis jarak cukup efektif untuk dataset ini.



Grafik menunjukkan akurasi model setelah tuning, di mana KNN tetap unggul dengan akurasi sekitar 80%, disusul Decision Tree (~75%), Naive Bayes (~62%), dan Logistic Regression yang paling rendah. Ini menegaskan bahwa KNN adalah model paling andal setelah proses tuning.



# Kesimpulan

Setelah melalui serangkaian tahapan, mulai dari Pemahaman Dataset, Eksplorasi Data dan Pra-pemrosesan, Implementasi Model, Evaluasi Model, hingga Analisis Hasil, kita dapat menyimpulkan bahwa KNN dan Decision Tree merupakan kandidat kuat untuk klasifikasi, sementara Logistic Regression dan Naive Bayes kurang cocok karena hubungan antar fitur yang tidak linier dan distribusi yang tidak normal. Proses standarisasi, tuning, dan evaluasi menyeluruh sangat penting untuk menghasilkan model yang andal dan memiliki generalisasi yang baik.

